# Simulación de Eventos Discretos (SED)

## Principios generales, fundamentos estocásticos y ejemplos manuales de colas

Alejandra Tabares Pozos

# Mapa de la sesión

1. ¿Qué es SED y cuándo usarla?
2. Relación con procesos estocásticos (trayectorias, eventos, cadenas embebidas).
3. Elementos constitutivos del modelado SED (entidades, recursos, eventos, estado, calendario).
4. Ejemplo manual paso a paso: cola con **un servidor**.
5. Ejemplo manual: cola con **dos servidores**.
6. **Worldviews** (event-scheduling, activity-scanning, process-interaction).

# Definición operativa (SED)

Un modelo de **simulación de eventos discretos** describe un sistema dinámico cuyo **estado** cambia en un conjunto *discreto* de instantes de tiempo llamados **eventos**. Entre eventos, el estado permanece constante (o evoluciona de forma determinista y controlada por reglas del modelo).

## Idea clave

SED genera **trayectorias muestrales** (sample paths) de un proceso de estado $X(t)$ y aproxima medidas de desempeño típicamente expresadas como esperanzas:

$$\theta = \mathbb{E}[g(X)] \quad \text{(p. ej. espera promedio, utilización, nivel de inventario).}$$

- Cambios *instantáneos* en el estado: llegadas, salidas, fallas, reparaciones, decisiones, etc.
- El tiempo avanza por **saltos** de un evento al siguiente (no necesariamente en pasos $\Delta t$ fijos).

# SED vs. simulación en tiempo discreto vs. dinámica continua

- **SED**: $X(t)$ es típicamente *càdlàg* (continua por la derecha con límites por la izquierda), con saltos en tiempos $T_1 < T_2 < \cdots$.
- **Tiempo discreto**: se observa/actualiza en $t = 0, \Delta t, 2\Delta t, \ldots$ (p. ej. modelos epidemiológicos discretizados, cadenas de Markov en tiempo discreto).
- **Continua**: dinámica descrita por EDO/SDE (p. ej. $dX_t = a(X_t)\,dt + b(X_t)\,dW_t$).

## Qué diferencia a SED

La granularidad temporal está inducida por la **lógica del sistema** (eventos) y no por una malla temporal impuesta.

# ¿Cuándo usar SED? (criterios de decisión)

Use SED cuando:

- Existe **interacción compleja** entre componentes (colas, recursos compartidos, prioridades, bloqueos).
- Hay **aleatoriedad estructural** (llegadas/servicios/fallas) y el análisis exacto es intratable.
- Importan **métricas no lineales** o condicionales (p. ej. percentiles, niveles de servicio, costo con penalizaciones).
- Se requieren **políticas operacionales** (reglas de despacho, asignación dinámica, control) difíciles de analizar en cerrado.

## No usar SED si...

- Un modelo analítico confiable existe y responde la pregunta (cola M/M/1 clásica, inventario simple, etc.).
- La pregunta es determinista/estática (optimización sin incertidumbre ni dinámica real).
- No hay datos/hipótesis mínimas para parametrizar la incertidumbre: simular no reemplaza modelar.

# SED como proceso estocástico: formalismo mínimo

Considere un **estado** $X(t) \in \mathcal{X}$ que resume toda la información necesaria para predecir el futuro (posiblemente ampliando el estado con *tiempos residuales*).

## Estructura típica

Existen tiempos de evento $0 = T_0 < T_1 < T_2 < \dots$ tales que:

$$X(t) = X(T_k) \quad \text{para } t \in [T_k, T_{k+1}),$$

y el salto $X(T_{k+1})$ se obtiene por una regla

$$X_{k+1} = \Phi(X_k,\, e_{k+1},\, U_{k+1}),$$

donde $X_k := X(T_k)$, $e_{k+1}$ denota el tipo de evento, y $U_{k+1}$ representa variables aleatorias generadas (p. ej. tiempos de servicio).

**Interpretación**: SED es una manera algorítmica de construir una trayectoria muestral de $X(t)$.

# Procesos puntuales y cadena embebida

- Los **tiempos de evento** $\{T_k\}$ inducen un **proceso puntual** (point process) y su **proceso contador**:

$$N(t) = \text{máx}\{k : \ T_k \leq t\}.$$

- La dinámica puede estudiarse en dos escalas:
  1. **En tiempo real** $t$: $X(t)$ (continuo en tiempo, con saltos).
  2. **En épocas de evento** $k$: $X_k = X(T_k)$ (tiempo discreto).

## Idea potente (cadena embebida)

En muchos sistemas, $\{X_k\}$ es (o puede hacerse) una **cadena de Markov en tiempo discreto** al definir un estado suficientemente rico. Esto conecta SED con teoría de colas, cadenas de Markov y procesos de renovación.

# Colas como procesos: $N(t)$, $Q(t)$ y métricas

En una cola, son procesos naturales:

$$N(t) = \text{número en el sistema}, \qquad Q(t) = \text{número en cola (esperando)}.$$

Para horizonte $[0, T]$, métricas típicas (promedios en el tiempo):

$$\bar{N}_T = \frac{1}{T}\int_0^T N(t)\,dt, \qquad \bar{Q}_T = \frac{1}{T}\int_0^T Q(t)\,dt.$$

Métricas por clientes (promedios sobre observaciones):

$$\bar{W}_n = \frac{1}{n}\sum_{i=1}^{n} W_i, \qquad \bar{T}_n = \frac{1}{n}\sum_{i=1}^{n}(D_i - A_i),$$

donde $A_i$ = llegada, $D_i$ = salida, $W_i$ = espera antes de servicio.

**Diferencia conceptual**

Promedios en el tiempo integran *áreas bajo trayectorias* $(N(t), Q(t))$. Promedios por cliente agregan *muestras por entidad*.

# Bloques de construcción de un modelo SED

(terminología estándar)

- **Entidades**: objetos que se mueven por el sistema (clientes, piezas, llamadas).
- **Atributos**: propiedades de entidades (clase, prioridad, tamaño, hora de llegada).
- **Recursos**: capacidad limitada que presta servicio (servidores, máquinas, operadores).
- **Colas**: buffers de espera asociados a recursos/políticas.
- **Estado**: variables mínimas para describir el sistema en $t$ (ocupación, longitudes, eventos programados).
- **Eventos**: ocurrencias instantáneas que cambian el estado (llegada, fin de servicio).
- **Actividades y demoras**:
  - Actividad: duración conocida/aleatoria que *se programa* (p. ej. servicio).
  - Demora: espera condicionada por estado (p. ej. espera en cola hasta que haya servidor).

## Corazón computacional

**Calendario de eventos (FEL)**: estructura que mantiene los próximos eventos con sus tiempos; típicamente una cola de prioridad por tiempo.

# Mecanismo de avance del tiempo: *next-event*

El esquema dominante en SED es el **avance al siguiente evento**:

1. Mantener una lista de eventos futuros con tiempos $\{t_e\}$.
2. Seleccionar $t^\star = \text{mín}_e\, t_e$ (evento más próximo).
3. Avanzar el reloj: $t \leftarrow t^\star$.
4. Ejecutar la rutina del evento: actualizar estado, recolectar estadísticas, programar nuevos eventos.

El reloj **salta** entre eventos
(no hay necesidad de iterar en pasos $\Delta t$).

# De la pregunta al experimento: ciclo de estudio de simulación

En Banks, el **estudio** de simulación (no solo el código) incluye:

- Formulación del problema y objetivos medibles.
- Conceptualización del modelo (supuestos, límites del sistema, métricas).
- Datos de entrada y modelado estocástico (distribuciones, dependencia, validación).
- Traducción a modelo computacional (worldview, estructura de eventos, recursos).
- Verificación (¿está bien implementado?) y validación (¿representa la realidad útilmente?).
- Diseño experimental: escenarios, políticas, sensibilidad, número de réplicas.
- Análisis de salida: estimación, intervalos de confianza, comparaciones.
- Documentación y comunicación de resultados.

**Mensaje**

El riesgo típico no es "programar mal", sino **preguntar mal**, **modelar mal la incertidumbre** o **interpretar mal la salida**.

# Ejemplo 1 (muy guiado): cola FCFS con 1 servidor

**Objetivo pedagógico:** ver *exactamente* cómo opera el enfoque **event-scheduling** de SED:

- Definir **estado** y **eventos**.
- Mantener una **FEL** (*future event list*) con los próximos tiempos de evento.
- Ejecutar el ciclo: **Initialization** → **Timing** → **Event routine** → **Update statistics**.

**Sistema**

Una cola única FCFS, un servidor, llegadas y servicios *dados* (muestra fija) para poder simular "a mano".

# Diccionario de métricas: por cliente vs por tiempo

## Tiempos por cliente (promedios sobre entidades)

Para cliente $i$: llegada $A_i$, inicio de servicio $B_i$, salida $D_i$.

$$W_i = B_i - A_i \ \text{(espera en cola)}, \qquad T_i = D_i - A_i \ \text{(tiempo en sistema)}.$$

Estimadores:

$$\bar{W}_n = \frac{1}{n}\sum_{i=1}^{n} W_i = \frac{\texttt{total_of_delays}}{\texttt{num_custs_delayed}}.$$

## Promedios en el tiempo (áreas bajo trayectorias)

$$\bar{Q}_T = \frac{1}{T}\int_0^T Q(t)\,dt = \frac{\texttt{area_num_in_q}}{T}, \qquad \hat{\rho} = \frac{1}{T}\int_0^T 1\{\text{BUSY}\}\,dt = \frac{\texttt{area_server_status}}{T}.$$

Con $c$ servidores: $\hat{\rho} = \frac{1}{c}\cdot\frac{\texttt{area_server_status}}{T}$ (donde `area_server_status` acumula $B(t) \in \{0,\ldots,c\}$).

# (Variables de estado, eventos y FEL

**Reloj:** $t$.

**Estado mínimo (en cada instante de evento):**

- $S \in \{\text{IDLE}, \text{BUSY}\}$: estado del servidor.
- $N_Q \in \mathbb{Z}_{\geq 0}$: número de clientes esperando en cola.
- `QUEUE`: lista FCFS (guardamos *tiempos de llegada* para calcular esperas).

**Eventos del modelo:**

- **ARRIVAL**: llega un cliente.
- **DEPARTURE**: termina un servicio (sale un cliente).

## FEL (Future Event List) como arreglo de tiempos

Para 1 servidor basta con mantener:

`time_next_event[1]` = tiempo de próxima llegada, `time_next_event[2]` = tiempo de próxima salida.

Si un evento no está programado, su tiempo es $\infty$.

# Datos de entrada (muestra fija) y qué "se genera" en ARRIVAL

Usaremos 6 clientes con tiempos entre llegadas $X_i$ y tiempos de servicio $S_i$:

| Cliente $i$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $X_i$ (entre llegadas) | 0.0 | 1.2 | 0.7 | 1.3 | 0.4 | 1.0 |
| $S_i$ (servicio) | 1.1 | 0.9 | 2.0 | 1.2 | 0.8 | 1.5 |

## Interpretación (para simular a mano)

En una simulación real, cada ARRIVAL haría:

- *Generar* el siguiente $X_{i+1}$ y programar la próxima llegada.
- *Generar* el servicio $S_i$ cuando el cliente **entra a servicio**.

Aquí, esos valores ya están listos en la tabla para poder calcular todo manualmente.

# Estructura: 4 rutinas (Initialization, Timing, ARRIVAL, DEPARTURE)

## Routine 1: Initialization

- $t \leftarrow 0$
- $S \leftarrow$ IDLE, $N_Q \leftarrow 0$, `QUEUE` vacía
- Programar 1ra llegada: `time_next_event[1]` $\leftarrow X_1$
- No hay salida programada: `time_next_event[2]` $\leftarrow \infty$

## Routine 2: Timing

- Elegir $e^* = \arg\min_e$ `time_next_event[e]`
- $t \leftarrow \min_e$ `time_next_event[e]`

## Routine 3: ARRIVAL

Al llegar cliente $i$ en tiempo $t$:

1. Programar próxima llegada (si existe):

$$\texttt{time_next_event[1]} \leftarrow t + X_{i+1}.$$

2. Si $S =$ IDLE: iniciar servicio **de inmediato**

$$S \leftarrow \text{BUSY}, \quad \texttt{time_next_event[2]} \leftarrow t + S_i.$$

3. Si $S =$ BUSY: encolar

$$N_Q \leftarrow N_Q + 1, \quad \text{enqueue}(t).$$

## Routine 4: DEPARTURE

En tiempo $t$:

1. Si $N_Q = 0$: $S \leftarrow$ IDLE, `time_next_event[2]` $\leftarrow \infty$
2. Si $N_Q 0$: tomar siguiente (FCFS), calcular demora

$$\text{delay} = t - \text{dequeue}(), \quad N_Q \leftarrow N_Q - 1,$$

iniciar nuevo servicio:

$$\texttt{time_next_event[2]} \leftarrow t + S_{\textbf{(siguiente)}}.$$

# Cómo se actualizan estadísticas por tiempo (áreas)

En SED, métricas promediadas en el tiempo se calculan con **áreas** entre eventos.

Defina:

$$\texttt{time_last_event}, \quad \texttt{area_num_in_q}, \quad \texttt{area_server_status}, \quad \texttt{total_delay}.$$

## Update-Time-Avg-Stats (en cada evento, antes de cambiar el estado)

Sea $\Delta = t - \texttt{time_last_event}$. Entonces:

$$\texttt{area_num_in_q} \mathrel{+}= N_Q \cdot \Delta, \qquad \texttt{area_server_status} \mathrel{+}= 1\{S = \text{BUSY}\} \cdot \Delta,$$

$$\texttt{time_last_event} \leftarrow t.$$

## Interpretación

`area_num_in_q` acumula el área bajo $N_Q(t)$ (cola), y `area_server_status` el área bajo el indicador "servidor ocupado".

# Bitácora (1/3): inicialización y primeros 4 eventos con FEL

Convención: FEL = (**NA** = next arrival, **ND** = next departure). Si no hay evento: tiempo = $\infty$.

| Evento # | $t$ | Evento | Estado servidor $S$ | $N_Q$ | FEL: NA | FEL: ND |
|---|---|---|---|---|---|---|
| Init | 0.0 | – | IDLE | 0 | 0.0 | $\infty$ |
| 1 | 0.0 | ARRIVAL(1) | BUSY | 0 | 1.2 | 1.1 |
| 2 | 1.1 | DEPARTURE(1) | IDLE | 0 | 1.2 | $\infty$ |
| 3 | 1.2 | ARRIVAL(2) | BUSY | 0 | 1.9 | 2.1 |
| 4 | 1.9 | ARRIVAL(3) | BUSY | 1 | 3.2 | 2.1 |

## Qué observar

En el Evento 4, el servidor está BUSY, así que el cliente 3 se **encola** ($N_Q$ sube a 1) y la ND no cambia.

# Bitácora (2/3): eventos 5 a 8 (cola creciendo y atención FCFS)

| Evento # | $t$ | Evento | $S$ | $N_Q$ | NA | ND |
|---|---|---|---|---|---|---|
| 5 | 2.1 | DEPARTURE(2) | BUSY | 0 | 3.2 | 4.1 |
| 6 | 3.2 | ARRIVAL(4) | BUSY | 1 | 3.6 | 4.1 |
| 7 | 3.6 | ARRIVAL(5) | BUSY | 2 | 4.6 | 4.1 |
| 8 | 4.1 | DEPARTURE(3) | BUSY | 1 | 4.6 | 5.3 |

## Detalles "Banks" dentro del DEPARTURE

En Evento 5 ($t$=2,1), $N_Q$=1 antes de atender: se hace dequeue y se calcula

$$\text{delay}_3 = 2{,}1 - 1{,}9 = 0{,}2,$$

luego se programa la nueva salida con el servicio del cliente 3: $2{,}1 + 2{,}0 = 4{,}1$.

# Bitácora (3/3): eventos 9 a 12 y condición de terminación

| Evento # | $t$ | Evento | $S$ | $N_Q$ | NA | ND |
|---|---|---|---|---|---|---|
| 9 | 4.6 | ARRIVAL(6) | BUSY | 2 | $\infty$ | 5.3 |
| 10 | 5.3 | DEPARTURE(4) | BUSY | 1 | $\infty$ | 6.1 |
| 11 | 6.1 | DEPARTURE(5) | BUSY | 0 | $\infty$ | 7.6 |
| 12 | 7.6 | DEPARTURE(6) | IDLE | 0 | $\infty$ | $\infty$ |

## Criterio de parada

Se termina cuando la FEL no tiene eventos finitos: NA=$\infty$ y ND=$\infty$.

# Ejemplo 1: tiempos por cliente (métricas por entidad)

A partir de la bitácora se reconstruyen, para cada cliente $i$:

$$A_i = \text{llegada}, \quad B_i = \text{inicio de servicio}, \quad D_i = \text{salida}, \qquad W_i = B_i - A_i, \quad T_i = D_i - A_i.$$

| $i$ | $A_i$ | $B_i$ | $D_i$ | $W_i$ | $T_i$ |
|---|---|---|---|---|---|
| **1** | **0.0** | **0.0** | **1.1** | **0.0** | **1.1** |
| **2** | **1.2** | **1.2** | **2.1** | **0.0** | **0.9** |
| **3** | **1.9** | **2.1** | **4.1** | **0.2** | **2.2** |
| **4** | **3.2** | **4.1** | **5.3** | **0.9** | **2.1** |
| **5** | **3.6** | **5.3** | **6.1** | **1.7** | **2.5** |
| **6** | **4.6** | **6.1** | **7.6** | **1.5** | **3.0** |
| **Suma** | | | | **4.3** | **11.8** |

## Promedios por cliente (una corrida)

$$\bar{W} = \frac{1}{6}\sum_{i=1}^{6} W_i = \frac{4{,}3}{6} \approx 0{,}717, \qquad \bar{T} = \frac{1}{6}\sum_{i=1}^{6} T_i = \frac{11{,}8}{6} \approx 1{,}967.$$

# Ejemplo 1: cálculo explícito de áreas y utilización (1 servidor)

Entre eventos, el estado es constante, así que las integrales de Banks se vuelven sumas:

$$\texttt{area_num_in_q} = \int_0^T N_Q(t)\,dt = \sum_k N_Q^{(k)}\,\Delta_k, \qquad \texttt{area_server_status} = \int_0^T B(t)\,dt = \sum_k B^{(k)}\,\Delta_k,$$

donde para 1 servidor $B(t) = 1\{S(t) = \text{BUSY}\} \in \{0,1\}$ y $\Delta_k = t_{k+1} - t_k$.

| **Intervalo** $[t_k,\, t_{k+1})$ | $\Delta_k$ | $N_Q^{(k)}$ | $B^{(k)}$ | $N_Q^{(k)}\Delta_k$ | $B^{(k)}\Delta_k$ |
|---|---|---|---|---|---|
| [0,0, 1,1) | 1.1 | 0 | 1 | 0.0 | 1.1 |
| [1,1, 1,2) | 0.1 | 0 | 0 | 0.0 | 0.0 |
| [1,2, 1,9) | 0.7 | 0 | 1 | 0.0 | 0.7 |
| [1,9, 2,1) | 0.2 | 1 | 1 | 0.2 | 0.2 |
| [2,1, 3,2) | 1.1 | 0 | 1 | 0.0 | 1.1 |
| [3,2, 3,6) | 0.4 | 1 | 1 | 0.4 | 0.4 |
| [3,6, 4,1) | 0.5 | 2 | 1 | 1.0 | 0.5 |
| [4,1, 4,6) | 0.5 | 1 | 1 | 0.5 | 0.5 |
| [4,6, 5,3) | 0.7 | 2 | 1 | 1.4 | 0.7 |
| [5,3, 6,1) | 0.8 | 1 | 1 | 0.8 | 0.8 |
| [6,1, 7,6) | 1.5 | 0 | 1 | 0.0 | 1.5 |
| **Suma** | $T = 7{,}6$ | | | 4.3 | 7.5 |

## Métricas (Banks: promedios en el tiempo)

$$\bar{Q}_T = \frac{\texttt{area_num_in_q}}{T} = \frac{4{,}3}{7{,}6} \approx 0{,}566, \qquad \hat{\rho} = \frac{\texttt{area_server_status}}{T} = \frac{7{,}5}{7{,}6} \approx 0{,}987.$$

Además, como $N(t) = N_Q(t) + B(t)$ en 1 servidor:

$$\bar{N}_T = \frac{1}{T}\int_0^T N(t)\,dt = \frac{4{,}3 + 7{,}5}{7{,}6} = \frac{11{,}8}{7{,}6} \approx 1{,}553.$$

# Ejemplo 2 (muy guiado): cola FCFS con 2 servidores y FEL

**Cambio estructural clave:** ahora hay **dos posibles eventos DEPARTURE**, uno por servidor.

## Estado mínimo

- $t$ (reloj), $N_Q$ (clientes esperando), `QUEUE` (tiempos de llegada).
- $S_1, S_2 \in \{\text{IDLE}, \text{BUSY}\}$ (estado de cada servidor).

## FEL como arreglo

$$\texttt{time_next_event[1]} = \text{próxima llegada (NA)},$$

$$\texttt{time_next_event[2]} = \text{próxima salida servidor 1 (ND1)}, \quad \texttt{time_next_event[3]} = \text{próxima salida servidor 2 (ND2)}.$$

Eventos no programados $\Rightarrow \infty$.

# Rutinas ARRIVAL y DEPARTURE con 2 servidores (regla exacta)

## ARRIVAL en tiempo $t$

1. Programar próxima llegada (si existe):
$$\texttt{NA} \leftarrow t + X_{i+1}.$$
2. Si hay servidor libre:
   - Si $S_1 = \text{IDLE}$: asignar a 1 y programar ND1 $= t + S_i$.
   - else si $S_2 = \text{IDLE}$: asignar a 2 y programar ND2 $= t + S_i$.
3. Si ambos BUSY: encolar
$$N_Q \leftarrow N_Q + 1, \quad \texttt{enqueue}(t).$$

## DEPARTURE del servidor $j \in \{1, 2\}$ en tiempo $t$

1. Si $N_Q = 0$:
$$S_j \leftarrow \text{IDLE}, \quad \texttt{ND}j \leftarrow \infty.$$
2. Si $N_Q > 0$:
$$\text{delay} = t - \texttt{dequeue}(), \quad N_Q \leftarrow N_Q - 1,$$
asignar ese cliente al servidor $j$ y programar:
$$\texttt{ND}j \leftarrow t + S_{(\text{siguiente})}.$$

## Timing

Elegir el menor entre NA, ND1 y ND2; avanzar $t$ y ejecutar la rutina asociada.

# Datos de entrada (muestra fija) para 2 servidores

| Cliente $i$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $X_i$ | 0.0 | 0.6 | 0.9 | 0.4 | 1.1 | 0.5 | 0.7 |
| $S_i$ | 1.4 | 1.0 | 2.2 | 0.8 | 1.6 | 1.3 | 0.9 |

## Inicialización (igual idea, más eventos)

$t \leftarrow 0,\ S_1 = S_2 = \text{IDLE},\ N_Q = 0;\ \text{NA} \leftarrow X_1 = 0{,}0;\ \text{ND1} \leftarrow \infty;\ \text{ND2} \leftarrow \infty.$

# Bitácora con FEL (1/2): eventos 1 a 7 (hasta que aparece cola)

FEL = (NA, ND1, ND2).

| # | $t$ | Evento | $(S_1, S_2)$ | $N_Q$ | NA | ND1 | ND2 |
|---|---|---|---|---|---|---|---|
| Init | 0.0 | – | (IDLE,IDLE) | 0 | 0.0 | $\infty$ | $\infty$ |
| 1 | 0.0 | ARR(1) | (BUSY,IDLE) | 0 | 0.6 | 1.4 | $\infty$ |
| 2 | 0.6 | ARR(2) | (BUSY,BUSY) | 0 | 1.5 | 1.4 | 1.6 |
| 3 | 1.4 | DEP(s1) | (IDLE,BUSY) | 0 | 1.5 | $\infty$ | 1.6 |
| 4 | 1.5 | ARR(3) | (BUSY,BUSY) | 0 | 1.9 | 3.7 | 1.6 |
| 5 | 1.6 | DEP(s2) | (BUSY,IDLE) | 0 | 1.9 | 3.7 | $\infty$ |
| 6 | 1.9 | ARR(4) | (BUSY,BUSY) | 0 | 3.0 | 3.7 | 2.7 |
| 7 | 2.7 | DEP(s2) | (BUSY,IDLE) | 0 | 3.0 | 3.7 | $\infty$ |

**Interpretación**

Hasta aquí, nunca hubo cola ($N_Q = 0$): siempre había al menos un servidor libre cuando llegaba alguien.

# Bitácora con FEL (2/2): eventos 8 a 14 (cola, demoras y asignación)

| # | $t$ | Evento | $(S_1, S_2)$ | $N_Q$ | NA | ND1 | ND2 |
|---|---|---|---|---|---|---|---|
| 8 | 3.0 | ARR(5) | (BUSY,BUSY) | 0 | 3.5 | 3.7 | 4.6 |
| 9 | 3.5 | ARR(6) | (BUSY,BUSY) | 1 | 4.2 | 3.7 | 4.6 |
| 10 | 3.7 | DEP(s1) | (BUSY,BUSY) | 0 | 4.2 | 5.0 | 4.6 |
| 11 | 4.2 | ARR(7) | (BUSY,BUSY) | 1 | $\infty$ | 5.0 | 4.6 |
| 12 | 4.6 | DEP(s2) | (BUSY,BUSY) | 0 | $\infty$ | 5.0 | 5.5 |
| 13 | 5.0 | DEP(s1) | (IDLE,BUSY) | 0 | $\infty$ | $\infty$ | 5.5 |
| 14 | 5.5 | DEP(s2) | (IDLE,IDLE) | 0 | $\infty$ | $\infty$ | $\infty$ |

## Dónde aparecen demoras (lo importante)

- Evento 9 (ARR(6)): ambos BUSY $\Rightarrow$ encola ($N_Q = 1$).
- Evento 10 (DEP(s1) en $t = 3{,}7$): toma al cliente 6 y su demora es $3{,}7 - 3{,}5 = 0{,}2$.
- Evento 11 (ARR(7)): ambos BUSY $\Rightarrow$ encola ($N_Q = 1$).
- Evento 12 (DEP(s2) en $t = 4{,}6$): toma al cliente 7 y su demora es $4{,}6 - 4{,}2 = 0{,}4$.

# Estadísticas con 2 servidores: áreas y utilización

La actualización por áreas es igual, pero ahora el "servidor ocupado" se mide como **número de servidores ocupados**.

Defina:

$$B(t) = 1\{S_1 = \text{BUSY}\} + 1\{S_2 = \text{BUSY}\} \in \{0, 1, 2\}.$$

## Update-Time-Avg-Stats (2 servidores)

Con $\Delta = t - \texttt{time_last_event}$:

$$\texttt{area_num_in_q} \mathrel{+}= N_Q \cdot \Delta, \qquad \texttt{area_server_status} \mathrel{+}= B(t) \cdot \Delta.$$

Entonces, en horizonte $T$:

$$\bar{Q}_T = \frac{\texttt{area_num_in_q}}{T}, \qquad \text{Utilización promedio por servidor} = \frac{1}{2} \cdot \frac{\texttt{area_server_status}}{T}.$$

# Ejemplo 2: cálculo explícito de áreas y utilización (2 servidores)

Entre eventos, el estado *no cambia*. Por tanto, las integrales de Banks se vuelven sumas por intervalos

$$\texttt{area_num_in_q} = \int_0^T N_Q(t)\,dt = \sum_k N_Q^{(k)}\,\Delta_k, \qquad \texttt{area_server_status} = \int_0^T B(t)\,dt = \sum_k B^{(k)}\,\Delta_k,$$

donde $B(t) = 1\{S_1 = \text{BUSY}\} + 1\{S_2 = \text{BUSY}\}$ y $\Delta_k = t_{k+1} - t_k$.

| **Intervalo** $[t_k, t_{k+1})$ | $\Delta_k$ | $N_Q^{(k)}$ | $B^{(k)}$ | $N_Q^{(k)}\Delta_k$ | $B^{(k)}\Delta_k$ |
|---|---|---|---|---|---|
| [0,0, 0,6) | 0.6 | 0 | 1 | 0.0 | 0.6 |
| [0,6, 1,4) | 0.8 | 0 | 2 | 0.0 | 1.6 |
| [1,4, 1,5) | 0.1 | 0 | 1 | 0.0 | 0.1 |
| [1,5, 1,6) | 0.1 | 0 | 2 | 0.0 | 0.2 |
| [1,6, 1,9) | 0.3 | 0 | 1 | 0.0 | 0.3 |
| [1,9, 2,7) | 0.8 | 0 | 2 | 0.0 | 1.6 |
| [2,7, 3,0) | 0.3 | 0 | 1 | 0.0 | 0.3 |
| [3,0, 3,5) | 0.5 | 0 | 2 | 0.0 | 1.0 |
| [3,5, 3,7) | 0.2 | 1 | 2 | 0.2 | 0.4 |
| [3,7, 4,2) | 0.5 | 0 | 2 | 0.0 | 1.0 |
| [4,2, 4,6) | 0.4 | 1 | 2 | 0.4 | 0.8 |
| [4,6, 5,0) | 0.4 | 0 | 2 | 0.0 | 0.8 |
| [5,0, 5,5) | 0.5 | 0 | 1 | 0.0 | 0.5 |
| **Suma** | $T = 5{,}5$ | | | 0.6 | 9.2 |

## Métricas (Banks: promedios en el tiempo)

$$\bar{Q}_T = \frac{\texttt{area_num_in_q}}{T} = \frac{0{,}6}{5{,}5} \approx 0{,}109, \qquad \overline{B} = \frac{\texttt{area_server_status}}{T} = \frac{9{,}2}{5{,}5} \approx 1{,}673,$$

$$\text{Utilización promedio por servidor} = \frac{\overline{B}}{2} = \frac{1}{2}\cdot\frac{9{,}2}{5{,}5} = \frac{4{,}6}{5{,}5} \approx 0{,}836.$$

# Puente con lo anterior: ya usamos *Event-Scheduling*

En los ejemplos manuales (1 servidor y 2 servidores) **ya implementamos** el worldview más clásico de Banks:

## Qué hicimos exactamente (mismo lenguaje de Banks)

- **Estado**: $N_Q$, estados de servidor(es), cola FCFS con tiempos de llegada.
- **FEL** (*Future Event List*) como arreglo `time_next_event[]`:
  - Ejemplo 1: NA (next arrival), ND (next departure).
  - Ejemplo 2: NA, ND1, ND2.
- **Reloj por mínimo**: rutina **Timing** $\Rightarrow t \leftarrow$ mín tiempos en FEL.
- **Rutinas de evento**: **Arrival** y **Departure** actualizan estado *y reprogram*an tiempos en FEL.
- **Estadísticas por áreas**: **Update-time-average-stats** entre eventos.

## Idea clave

Un *worldview* no cambia el sistema que modelas: cambia **cómo organizas la lógica** para construir la misma trayectoria muestral.

# ¿Qué significa *worldview*?

**formas estándar de representar y programar** modelos SED.

## Un mismo sistema, tres formas de "contarlo"

- **Event-Scheduling**: "cuándo ocurre el próximo evento" (agenda explícita: FEL).
- **Activity-Scanning**: "qué actividades están habilitadas ahora" (condiciones que se vuelven verdaderas).
- **Process-Interaction**: "qué hace cada entidad" (procesos con esperas por recursos).

## Criterio para elegir

**Legibilidad vs. control**:

- Event-scheduling: máximo control del reloj y la agenda (muy cercano a la teoría).
- Process-interaction: máxima legibilidad (el motor traduce esperas a eventos).

# Los tres *worldviews*

## 1) Event-Scheduling (visión de eventos)

Se escribe un **programa por rutinas**: **Initialization**, **Timing**, y rutinas de evento (**Arrival**, **Departure**, ...). La **FEL** guarda los tiempos de los próximos eventos (p. ej. NA, ND1, ND2).

## 2) Activity-Scanning (visión de actividades/condiciones)

Se escribe un conjunto de **actividades** y sus **condiciones de inicio**. El motor *escanea* condiciones y activa actividades cuando son verdaderas. (Útil cuando dominan reglas lógicas.)

## 3) Process-Interaction (visión de procesos)

Se escribe el **flujo de una entidad** (llega $\rightarrow$ espera $\rightarrow$ toma recurso $\rightarrow$ sirve $\rightarrow$ sale). Las "esperas" generan eventos internamente. (Ej.: SimPy.)

# Sistema común a los 3 worldviews: cola FCFS con 1 servidor

## Sistema (fijamos el mismo modelo para comparar)

- Una cola FCFS, un servidor con capacidad 1.
- Llegadas en tiempos $A_1 < A_2 < \cdots$ (p.ej. renovación / Poisson).
- Servicios $S_1, S_2, \ldots$ (p.ej. i.i.d.), no preemptivo.

## Estado mínimo (equivalente en cualquier worldview)

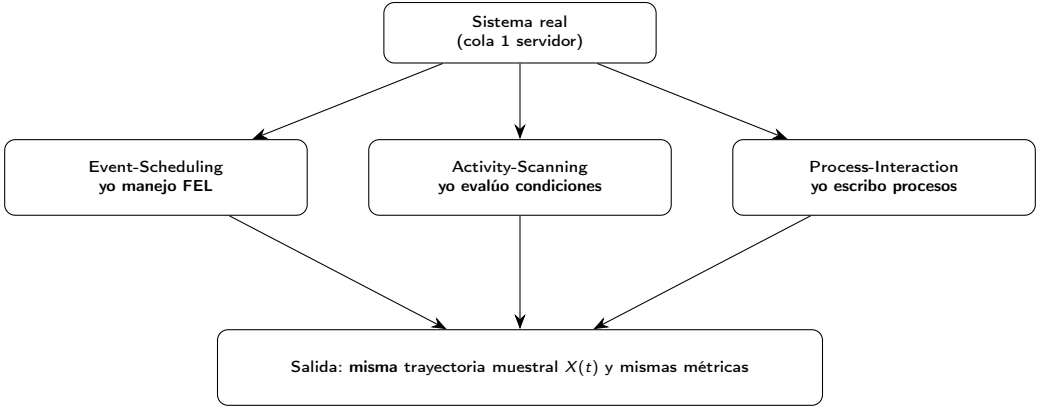
$$X(t) = (N_Q(t),\, S(t)), \quad S(t) \in \{\text{IDLE}, \text{BUSY}\}, \quad N_Q(t) \in \mathbb{Z}_{\geq 0}.$$

## Métricas (mismas para los 3)

$$\bar{W}_n = \frac{1}{n}\sum_{i=1}^{n} W_i, \qquad \bar{Q}_T = \frac{1}{T}\int_0^T N_Q(t)\,dt, \qquad \hat{\rho} = \frac{1}{T}\int_0^T 1\{S(t) = \text{BUSY}\}\,dt.$$

# Idea central: mismo sistema $\Rightarrow$ misma trayectoria, distinta "forma de escribirla"

Los tres worldviews son **representaciones computacionales** diferentes de la misma dinámica.




**Mensaje didáctico**

No es "tres teorías": es **tres interfaces** para programar el mismo motor lógico (eventos + estado + tiempo).

# Worldview 1: Event-Scheduling (lo que ya hicimos a mano)

**¿Qué escribes?** Rutinas de evento + una **FEL** explícita.

## FEL mínima (1 servidor)

`time_next_event[1]` = NA (próxima llegada), `time_next_event[2]` = ND (próxima salida).

## Ciclo (Banks): Timing + rutinas

```
Initialization();
while (not stop) {
  Timing();                  // clock <- min(NA, ND)
  Update_time_avg_stats();   // áreas: N_Q y BUSY
  if (next_event == ARRIVAL)   Arrival();
  else                         Departure();
}
Report();
```

## Intuición

Tú decides "**qué evento ocurre después**" porque tú mantienes la agenda (FEL).

## Worldview 2: Activity-Scanning (misma cola, pero pensando en "actividades habilitadas")

**¿Qué escribes?** Una lista de **actividades** y sus **condiciones de inicio**. En vez de "rutinas de evento", la lógica central es: *¿qué puede empezar ahora?*

### Actividades para la cola 1 servidor

- **A1: Iniciar servicio** (StartService)

  $$\text{Condición: } (S = \text{IDLE}) \ \wedge \ (N_Q > 0).$$

  Efecto: sacar 1 de cola, poner $S = \text{BUSY}$ y fijar un tiempo de fin de servicio.
- **A2: Llegada** (Arrival) puede tratarse como "actividad exógena" que incrementa $N_Q$ en su tiempo de ocurrencia.

### Idea de algoritmo (esqueleto)

```
Initialization();
while (not stop) {

  Advance_time();     // versión simple: reloj avanza y se revisa
  Execute_exogenous(); // llegadas que ocurren en este instante

  while (StartService condition is true) {
     StartService();  // escaneo + ejecución mientras haya actividades habilitadas
  }

  Update_statistics();
}
```

### Por qué confunde a estudiantes

# Activity-Scanning: cómo "se ve" en la cola 1 servidor (mini-ejemplo lógico)

Imagina que estás justo después de una salida (fin de servicio) en tiempo $t$:

## Estado al inicio del instante $t$

$$S(t^-) = \text{BUSY} \rightarrow S(t) = \text{IDLE}, \qquad N_Q(t) = 2.$$

En **Event-Scheduling**, la rutina **Departure** inmediatamente hace:

$$\text{si } N_Q > 0 \Rightarrow \text{toma siguiente y programa ND} = t + S_{\text{siguiente}}.$$

En **Activity-Scanning**, el motor haría conceptualmente:

$$\underbrace{(S = \text{IDLE}) \wedge (N_Q > 0)}_{\text{condición StartService}} = \text{TRUE} \Rightarrow \text{ejecutar StartService.}$$

Tras ejecutar StartService una vez:

$$S = \text{BUSY},\ N_Q = 1 \quad \Rightarrow \quad \text{condición StartService ahora es FALSE (porque } S \neq \text{IDLE).}$$

## Lo esencial

El "**if**" está invertido:

- Event-scheduling: **evento** dispara lógica.
- Activity-scanning: **estado** habilita lógica.

# Worldview 3: Process-Interaction (misma cola, pero escribiendo la "vida del cliente")

**¿Qué escribes?** Un **proceso por entidad** (cliente) que hace: llegar → esperar recurso → servir → salir.

## Pseudocódigo estilo SimPy (conceptual)

```
process Customer(i):
   arrive at time A_i
   enqueue (implicitly by requesting the server)

   request server (capacity=1)      // waits until available
   start service immediately when granted
   hold for service time S_i        // a timeout activity
   release server and depart
```

## Qué pasa "debajo del capó"

Aunque tú no veas una FEL, el motor:

- agenda internamente los eventos de "timeout" (fin de servicio),
- maneja la cola del recurso (FCFS),
- avanza el reloj al próximo evento (next-event).

# Process-Interaction aplicado al Ejemplo 1: qué cambia y qué no

## Qué NO cambia (mismo modelo)

- Sistema: una cola FCFS, un servidor (capacidad 1).
- Datos (muestra fija para correr a mano): llegadas $A_i$ y servicios $S_i$ del Ejemplo 1.
- Métricas: $\bar{W}$, $\bar{T}$, $\bar{Q}_T$, utilización $\hat{\rho}$.

## Qué SÍ cambia (forma de programar la lógica)

En lugar de escribir rutinas **Arrival/Departure** y mantener una FEL explícita, se escribe:

- un **proceso por cliente** (su "ciclo de vida"),
- un **recurso** (servidor) con cola FCFS interna,
- y **esperas** expresadas como "pedir recurso" y "timeout".

## Mensaje didáctico

Process-interaction *no elimina* los eventos; los **genera internamente** a partir de `request`, `release` y `timeout`.

# Estado del sistema en process-interaction: estado implícito y equivalente

En process-interaction, el estado que antes escribías como $(S(t), N_Q(t))$ se reconstruye desde el recurso:

## Recurso servidor (capacidad 1)

- **Ocupación**: `server.count` $\in \{0, 1\}$ (número en servicio).
- **Cola interna**: `len(server.queue)` $= N_Q(t)$ (número esperando).

## Equivalencias exactas (para 1 servidor)

$$B(t) = 1\{S(t) = \text{BUSY}\} \equiv \texttt{server.count}, \qquad N_Q(t) \equiv \texttt{len(server.queue)}.$$

y por tanto

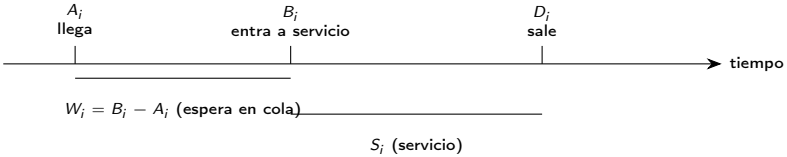
$$N(t) = N_Q(t) + B(t) \equiv \texttt{len(server.queue)} + \texttt{server.count}.$$

## Lo importante

El estudiante debe ver que "estado implícito" no significa "estado inexistente": está en la estructura del recurso.

# Ciclo de vida del cliente $i$: tiempos $A_i, B_i, D_i$ sin rutina Departure

En process-interaction, cada cliente es un proceso que "vive" esta secuencia:




## Definiciones (métricas por cliente)

$$W_i = B_i - A_i, \qquad T_i = D_i - A_i = (B_i - A_i) + S_i.$$

## Interpretación

En process-interaction, $B_i$ es el instante en que el **request** es concedido (evento interno "grant").

# Ejemplo 1 en process view: misma corrida, reconstrucción de $A_i, B_i, D_i$

Usamos los mismos datos del Ejemplo 1:

$$A = (0{,}0,\ 1{,}2,\ 1{,}9,\ 3{,}2,\ 3{,}6,\ 4{,}6), \quad S = (1{,}1,\ 0{,}9,\ 2{,}0,\ 1{,}2,\ 0{,}8,\ 1{,}5).$$

## Regla FCFS (1 servidor) que "emerge" del recurso

Si el servidor está libre al llegar, $B_i = A_i$; si no, el cliente espera hasta que el servidor se libere. En esta cola, eso equivale a la recurrencia:

$$B_1 = A_1, \qquad B_i = \text{máx}\{A_i,\ D_{i-1}\}, \qquad D_i = B_i + S_i.$$

## Con estos datos se obtiene (misma bitácora)

$$B = (0{,}0,\ 1{,}2,\ 2{,}1,\ 4{,}1,\ 5{,}3,\ 6{,}1), \qquad D = (1{,}1,\ 2{,}1,\ 4{,}1,\ 5{,}3,\ 6{,}1,\ 7{,}6).$$

## Punto pedagógico

En event-scheduling tú calculas $B_i$ "a mano" con `dequeue`; en process-interaction el recurso lo hace.

# Bajo el capó: qué eventos está creando el motor (aunque tú no los escribas)

Process-interaction sigue siendo **next-event**.

## Tres "operaciones" $\Rightarrow$ tres tipos de eventos internos

- `env.timeout(x)`: agenda un evento en $t + x$ (ej.: fin de servicio).
- `server.request()`: crea un evento "grant" que ocurre:
  - inmediatamente si hay capacidad,
  - o en el instante en que algún `release` libera capacidad.
- `release`: dispara la activación del próximo request en la cola FCFS del recurso.

## Correspondencia con la FEL del Ejemplo 1

- Los **ARRIVAL** los crea el generador de llegadas (o el `timeout` hasta $A_i$).
- Los **DEPARTURE** los crea cada cliente cuando hace `timeout(service_time)`.

## Mensaje

La FEL existe, solo que ahora es un detalle de implementación del motor.

# Cómo recolectar métricas en process view: por cliente y por tiempo

## (I) Métricas por cliente: timestamps

Cada cliente registra:

$$A_i = \text{arrival_time}, \quad B_i = \text{time cuando se concede el request}, \quad D_i = \text{time al terminar servicio}.$$

y se acumula:

$$\texttt{total_of_delays} \mathrel{+}= W_i, \qquad \texttt{sum_system_time} \mathrel{+}= T_i.$$

## (II) Métricas promediadas en el tiempo: áreas por monitoreo

Necesitas un **monitor** que acumule áreas cuando cambia el estado:

$$\texttt{area_num_in_q} = \int_0^T N_Q(t)\,dt, \qquad \texttt{area_server_status} = \int_0^T B(t)\,dt.$$

Con 1 servidor:

$$N_Q(t) = \texttt{len(server.queue)}, \qquad B(t) = \texttt{server.count}.$$

## Idea práctica

El motor no "te da" áreas por defecto: tú decides cuándo muestrear o, mejor, cuándo registrar cambios de estado.

# Process View: la tabla por cliente

En **Process View** no llevas una FEL; llevas una **tabla por cliente** (entity lifecycle): cada fila es un cliente, y se calcula cuándo empieza/termina su servicio.

## Columnas estándar (como el process table)

| $i$ | $X_i$ | $A_i$ | $S_i$ |
|---|---|---|---|
| | | $E_i$ | $F_i$ |
| | | $W_i$ | $T_i$ |
| | | $I_i$ | |

donde:

$A_i$ = Arrival, $E_i$ = Service Begin, $F_i$ = Service End, $W_i$ = Time in Queue, $T_i$ = Time in System, $I_i$ = Idle Time.

# Inputs del Ejemplo 1: interarrivals y service times

Usamos exactamente los datos del Ejemplo 1 (muestra fija):

| Cliente $i$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $X_i$ (entre llegadas) | 0.0 | 1.2 | 0.7 | 1.3 | 0.4 | 1.0 |
| $S_i$ (servicio) | 1.1 | 0.9 | 2.0 | 1.2 | 0.8 | 1.5 |

**Primero generas la secuencia de llegadas**

$$A_1 = 0, \qquad A_i = A_{i-1} + X_i \ \ (i \geq 2).$$

# Regla dorada (Process View): cuándo empieza el servicio

Esto es TODO el "motor" del process view (la lámina del MAX):

**Inicio de servicio**

$$E_1 = A_1, \qquad E_i = \text{máx}\{A_i,\ F_{i-1}\} \quad (i \geq 2).$$

**Fin de servicio**

$$F_i = E_i + S_i.$$

**Derivados (métricas por cliente)**

$$W_i = E_i - A_i, \qquad T_i = F_i - A_i, \qquad I_i = \text{máx}\{0,\ E_i - F_{i-1}\}\ (i \geq 2),\ \ I_1 = 0.$$

# Paso 1–2: construir $A_i$ (arrival stream) y completar cliente 1

## Llegadas (sumando interarrivals)

$$A = (0{,}0,\ 1{,}2,\ 1{,}9,\ 3{,}2,\ 3{,}6,\ 4{,}6).$$

## Cliente 1 (sistema vacío $\Rightarrow$ servicio inmediato)

$$E_1 = A_1 = 0{,}0, \qquad F_1 = E_1 + S_1 = 0{,}0 + 1{,}1 = 1{,}1,$$
$$W_1 = 0, \qquad T_1 = 1{,}1, \qquad I_1 = 0.$$

# Paso 3: caso "servidor ocioso" (cliente 2)

Cliente 2 llega cuando el servidor ya terminó con el cliente 1:

$$A_2 = 1{,}2, \qquad F_1 = 1{,}1 \;\Rightarrow\; \text{servidor libre al llegar.}$$

## Cálculo fila 2

$$E_2 = \text{máx}(A_2, F_1) = \text{máx}(1{,}2, 1{,}1) = 1{,}2,$$
$$F_2 = E_2 + S_2 = 1{,}2 + 0{,}9 = 2{,}1,$$
$$W_2 = E_2 - A_2 = 0, \qquad T_2 = F_2 - A_2 = 0{,}9,$$
$$I_2 = E_2 - F_1 = 1{,}2 - 1{,}1 = 0{,}1.$$

## Lectura

Hubo **0.1** unidades de tiempo con el servidor **ocioso** entre 1,1 y 1,2.

# Paso 4: "se forma cola" (cliente 3) y regla MAX en acción

Cliente 3 llega antes de que el cliente 2 termine:

$$A_3 = 1{,}9, \qquad F_2 = 2{,}1 \;\Rightarrow\; \text{servidor ocupado al llegar.}$$

## Cálculo fila 3

$$E_3 = \text{máx}(A_3, F_2) = \text{máx}(1{,}9, 2{,}1) = 2{,}1, \qquad F_3 = E_3 + S_3 = 2{,}1 + 2{,}0 = 4{,}1,$$
$$W_3 = E_3 - A_3 = 2{,}1 - 1{,}9 = 0{,}2, \qquad T_3 = F_3 - A_3 = 4{,}1 - 1{,}9 = 2{,}2,$$
$$I_3 = \text{máx}(0, E_3 - F_2) = 0.$$

## Lectura

Aquí aparece la cola: el cliente 3 **espera** 0,2 antes de entrar.

# Completar clientes 4–6 (misma receta, fila por fila)

Se repite exactamente la misma receta:

## Cliente 4

$$E_4 = \text{máx}(A_4, F_3) = \text{máx}(3{,}2, 4{,}1) = 4{,}1, \quad F_4 = 4{,}1 + 1{,}2 = 5{,}3, \quad W_4 = 0{,}9, \quad T_4 = 2{,}1.$$

## Cliente 5

$$E_5 = \text{máx}(A_5, F_4) = \text{máx}(3{,}6, 5{,}3) = 5{,}3, \quad F_5 = 5{,}3 + 0{,}8 = 6{,}1, \quad W_5 = 1{,}7, \quad T_5 = 2{,}5.$$

## Cliente 6

$$E_6 = \text{máx}(A_6, F_5) = \text{máx}(4{,}6, 6{,}1) = 6{,}1, \quad F_6 = 6{,}1 + 1{,}5 = 7{,}6, \quad W_6 = 1{,}5, \quad T_6 = 3{,}0.$$

# Tabla completa (Process View) — Ejemplo 1 (6 clientes)

| Cliente $i$ | $X_i$ | $A_i$ | $S_i$ | $E_i$ (begin) | $F_i$ (end) | $W_i$ | $T_i$ | $I_i$ |
|---|---|---|---|---|---|---|---|---|
| 1 | 0.0 | 0.0 | 1.1 | 0.0 | 1.1 | 0.0 | 1.1 | 0.0 |
| 2 | 1.2 | 1.2 | 0.9 | 1.2 | 2.1 | 0.0 | 0.9 | 0.1 |
| 3 | 0.7 | 1.9 | 2.0 | 2.1 | 4.1 | 0.2 | 2.2 | 0.0 |
| 4 | 1.3 | 3.2 | 1.2 | 4.1 | 5.3 | 0.9 | 2.1 | 0.0 |
| 5 | 0.4 | 3.6 | 0.8 | 5.3 | 6.1 | 1.7 | 2.5 | 0.0 |
| 6 | 1.0 | 4.6 | 1.5 | 6.1 | 7.6 | 1.5 | 3.0 | 0.0 |
| **Suma** | | | **7.5** | | | **4.3** | **11.8** | **0.1** |

# Métricas (directo desde la tabla, sin áreas)

De la tabla (una corrida):

## Promedios por cliente

$$\bar{W} = \frac{\sum W_i}{n} = \frac{4{,}3}{6} \approx 0{,}717, \qquad \bar{T} = \frac{\sum T_i}{n} = \frac{11{,}8}{6} \approx 1{,}967.$$

## Probabilidad de esperar

$$\widehat{\mathbb{P}}(W > 0) = \frac{\#\{i : W_i > 0\}}{n} = \frac{4}{6} \approx 0{,}667.$$

## Utilización del servidor (process view)

$$\hat{\rho} = \frac{\text{Total tiempo en servicio}}{\text{Tiempo total observado}} = \frac{\sum S_i}{F_6 - 0} = \frac{7{,}5}{7{,}6} \approx 0{,}987.$$

# Tabla comparativa: misma cola 1 servidor, tres formas de modelar

| | **Event-Scheduling** | **Activity-Scanning** | **Process-Interaction** |
|---|---|---|---|
| **Unidad de lógica** | Evento (Arrival/Departure) | Condición/actividad | Proceso de entidad |
| **Qué escribes** | FEL + rutinas | Reglas "si condición entonces" | Flujo del cliente + recursos |
| **Tiempo avanza** | mín en FEL (explícito) | por escaneo (a veces por incrementos) | mín en agenda interna |
| **Ventaja** | Control + transparencia teórica | Natural con reglas lógicas | Máxima legibilidad (alto nivel) |
| **Riesgo** | Verboso; fácil equivocarse en FEL | Ineficiente si time-slicing; difícil de depurar | "Magia": el estudiante no ve FEL |

## Recomendación para pregrado

Primero **Event-Scheduling** (para entender estado/FEL/estadísticas), luego **Process-Interaction** (para modelar grande en Python).

# Mapa de correspondencias: cómo se traducen las mismas ideas

## Misma cola 1 servidor: equivalencias conceptuales

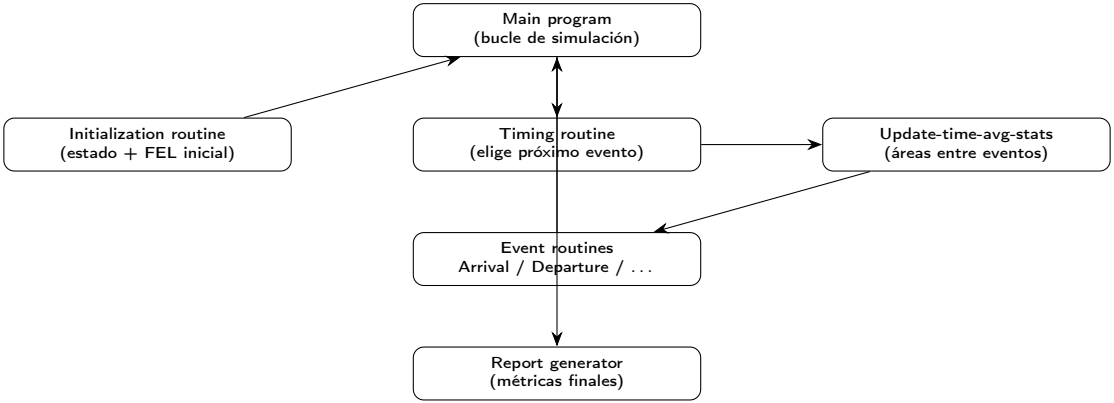
- **FEL (Event-Scheduling)** $\leftrightarrow$ **agenda interna** (Process-Interaction) (ambas contienen "próximo ARRIVAL" y "próximo fin de servicio").
- **Departure routine** $\leftrightarrow$ **release del recurso** + activación del siguiente en cola.
- **Si servidor IDLE al llegar** $\leftrightarrow$ **request se concede inmediatamente**.
- **Cola explícita** `time_arrive[ ]` $\leftrightarrow$ **cola del recurso** (SimPy) + registro de timestamps.

## Qué debes exigir en cualquier worldview (criterio de rigor)

1. Estado bien definido (suficiente para "predecir").
2. Reglas de transición claras (qué cambia en cada evento/actividad/paso).
3. Estadísticas bien definidas (por tiempo vs por cliente).

# Worldview 1: arquitectura estándar de un simulador Event-Scheduling

Banks presenta una **plantilla** (program structure) que separa responsabilidades:




## Qué gana el estudiante

Esta estructura obliga a: **(i)** definir estado y FEL con precisión, **(ii)** separar "avanzar reloj" de "cambiar estado", **(iii)** calcular estadísticas por áreas de forma correcta.

# Ejemplo completo (1/4): problema, estado y FEL (1 servidor, FCFS)

**Problema:** simular una cola de **1 servidor** FCFS y estimar:

$$\text{(i) espera promedio en cola } \bar{W}_q, \quad \text{(ii) } \bar{Q} = \frac{1}{T}\int_0^T N_Q(t)\,dt, \quad \text{(iii) utilización } \hat{\rho}.$$

## Estado y estructuras (mínimo necesario)

- `clock` $= t$
- `server_status` $\in \{\text{IDLE}, \text{BUSY}\}$
- `num_in_q` $= N_Q$
- `time_arrive[]`: arreglo (cola FCFS) con tiempos de llegada de quienes esperan

## FEL como arreglo `time_next_event`

`time_next_event[1]` $=$ NA (próxima llegada), `time_next_event[2]` $=$ ND (próxima salida).

Si no hay salida programada: `time_next_event[2]` $= \infty$.

# Ejemplo completo (2/4): acumuladores estadísticos (áreas) y criterio de parada

## Acumuladores

- `time_last_event`
- `total_of_delays` $= \sum$ esperas en cola (por cliente)
- `num_custs_delayed` = número de clientes que *entran a servicio* (para promediar)
- `area_num_in_q` $= \int N_Q(t)\,dt$
- `area_server_status` $= \int 1\{\text{BUSY}\}\,dt$

## Rutina: Update-time-average-stats

Con $\Delta = \texttt{clock} - \texttt{time_last_event}$:

$$\texttt{area_num_in_q} \mathrel{+}= \texttt{num_in_q} \cdot \Delta, \qquad \texttt{area_server_status} \mathrel{+}= 1\{\texttt{server_status=BUSY}\} \cdot \Delta,$$

$$\texttt{time_last_event} \leftarrow \texttt{clock}.$$

## Criterio de parada

Se suele fijar `num_delays_required` $= N$ y parar cuando

$$\texttt{num_custs_delayed} \geq \texttt{num_delays_required}.$$

# Ejemplo completo (3/4): Main program + Timing routine (pseudocódigo Banks)

## Main program (estructura canónica)

```
Initialization();

while (num_custs_delayed < num_delays_required) {

  Timing();                  // clock <- next event time, next_event_type <- argmin

  Update_time_avg_stats();   // áreas entre el último evento y éste

  if (next_event_type == ARRIVAL)   Arrival();
  else if (next_event_type == DEPARTURE) Departure();
}

Report();                    // métricas finales
```

## Timing routine (elige el mínimo en la FEL)

```
Timing() {
  min_time_next_event = infinity;
  next_event_type = 0;

  for (i = 1; i <= num_events; i++) {
    if (time_next_event[i] < min_time_next_event) {
      min_time_next_event = time_next_event[i];
      next_event_type = i;
```

# Ejemplo completo (4/4): Arrival, Departure, Initialization y Report

## Initialization (idea Banks)

```
Initialization() {
  clock = 0.0;
  server_status = IDLE;
  num_in_q = 0;
  time_last_event = 0.0;

  total_of_delays = 0.0;
  num_custs_delayed = 0;
  area_num_in_q = 0.0;
  area_server_status = 0.0;

  time_next_event[ARRIVAL]   = clock + expon(mean_interarrival);
  time_next_event[DEPARTURE] = infinity;
}
```

## Arrival (Banks: programa siguiente llegada)

```
Arrival() {
  time_next_event[ARRIVAL] = clock + expon(mean_interarrival);

  if (server_status == BUSY) {
    num_in_q++;
    time_arrive[num_in_q] = clock;        // enqueue arrival time
  } else {
    delay = 0.0;
    total_of_delays += delay;

    num_custs_delayed++;
```

## Departure (Banks: si hay cola, saca FCFS y calcula delay)

```
Departure() {
  if (num_in_q == 0) {
    server_status = IDLE;
    time_next_event[DEPARTURE] = infinity;
  } else {
    num_in_q--;
    delay = clock - time_arrive[1];
    total_of_delays += delay;

    num_custs_delayed++;

    for (i = 1; i <= num_in_q; i++)
      time_arrive[i] = time_arrive[i+1]; // shift left (FCFS)

    time_next_event[DEPARTURE] = clock + expon(mean_service);
  }
}
```

## Report (métricas Banks)

```
Report() {
  avg_delay_in_q = total_of_delays / num_custs_delayed;
  avg_num_in_q   = area_num_in_q / clock;
  util_server    = area_server_status / clock;
}
```

# Referencia base

J. Banks, J. S. Carson II, B. L. Nelson, D. M. Nicol.
*Discrete-Event System Simulation*.